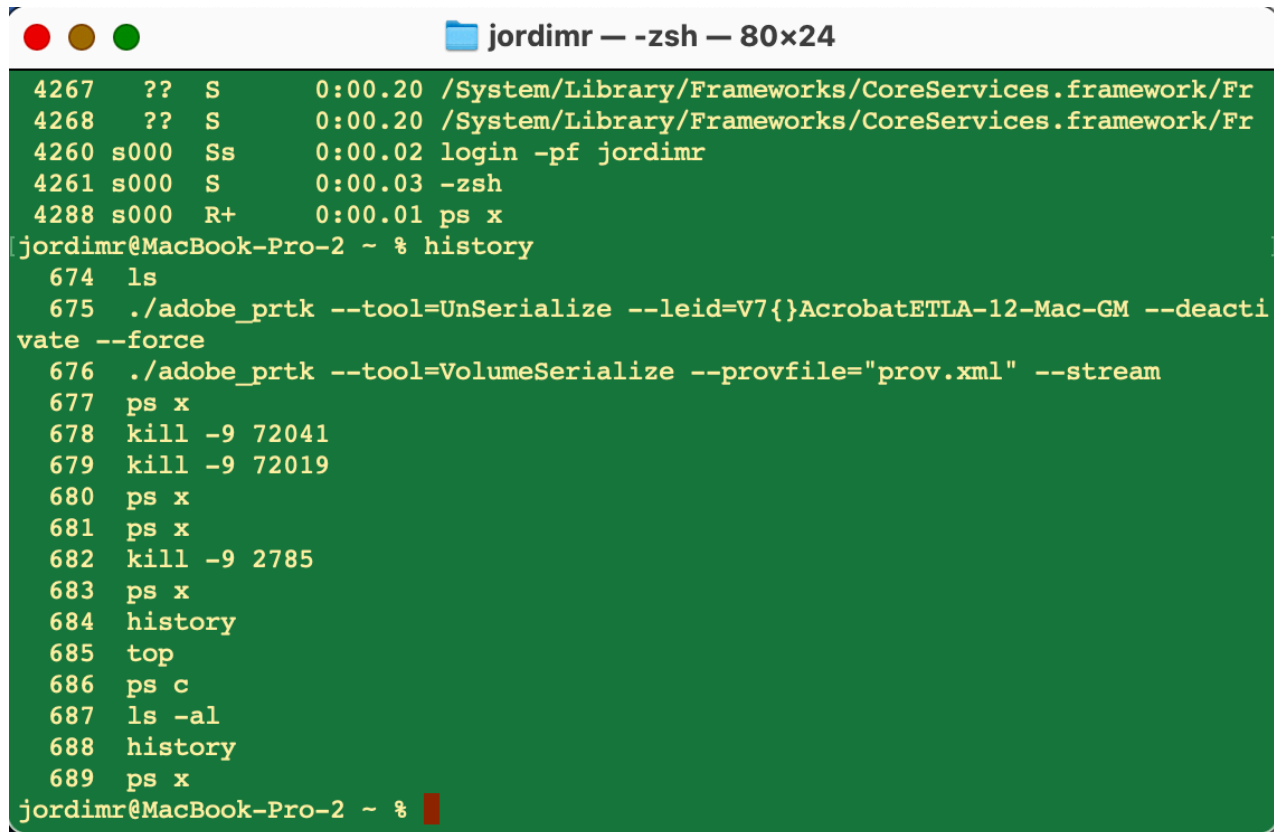


1. Basics of Linux

High level Operating System for general purposes. A few simple steps:

1. Open a **Terminal** window
2. Start writing 'history' in the command line (marked with >): you'll get...



```
jordimr — -zsh — 80x24
4267  ??  S      0:00.20 /System/Library/Frameworks/CoreServices.framework/Fr
4268  ??  S      0:00.20 /System/Library/Frameworks/CoreServices.framework/Fr
4260 s000 Ss     0:00.02 login -pf jordimr
4261 s000 S      0:00.03 -zsh
4288 s000 R+    0:00.01 ps x
jordimr@MacBook-Pro-2 ~ % history
674  ls
675  ./adobe_prtk --tool=UnSerialize --leid=V7{}AcrobatETLA-12-Mac-GM --deacti
vate --force
676  ./adobe_prtk --tool=VolumeSerialize --provfile="prov.xml" --stream
677  ps x
678  kill -9 72041
679  kill -9 72019
680  ps x
681  ps x
682  kill -9 2785
683  ps x
684  history
685  top
686  ps c
687  ls -al
688  history
689  ps x
jordimr@MacBook-Pro-2 ~ %
```

Useful LINUX commands: Try them!

0. > **whoami**: indeed...

1. **pwd**: where (in which folder) I am

2. **cd**: call a directory/folder

Try: > pwd

/home/usr/

> cd /

/

3. **mkdir** *new_dir*: create the directory *new_dir*

4. **rmdir** *new_dir*: remove the directory *new_dir*

5. **ls**: list of files in current folder

6. **ls -al**: list with characteristics/types of all files

7. **exit**: leave terminal/shell session. **logout** works same way.

8. **passwd**: change your password

9. **cp (-r)**: copy files (or folders)

Example: 'cp /etc/work/file1.dat /home/usr/new_file.txt'

or: 'cp /etc/work/file1.dat .'

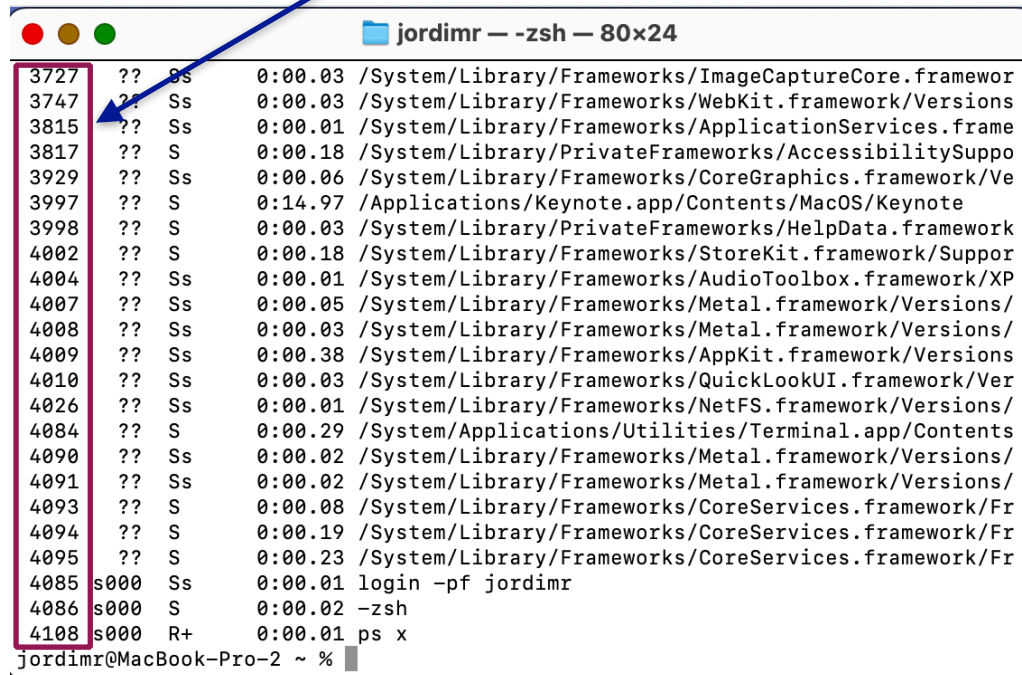
10. **rm (-r)**: remove a file (folder). Example: 'rm file1.dat'

11. **mv**: move files between directories.

Example: 'mv /etc/work/file1.dat /home/usr/new_file.txt'. The former file1.dat in directory /etc/work/ no longer exists...

12. **top**: list all processes running in the computer

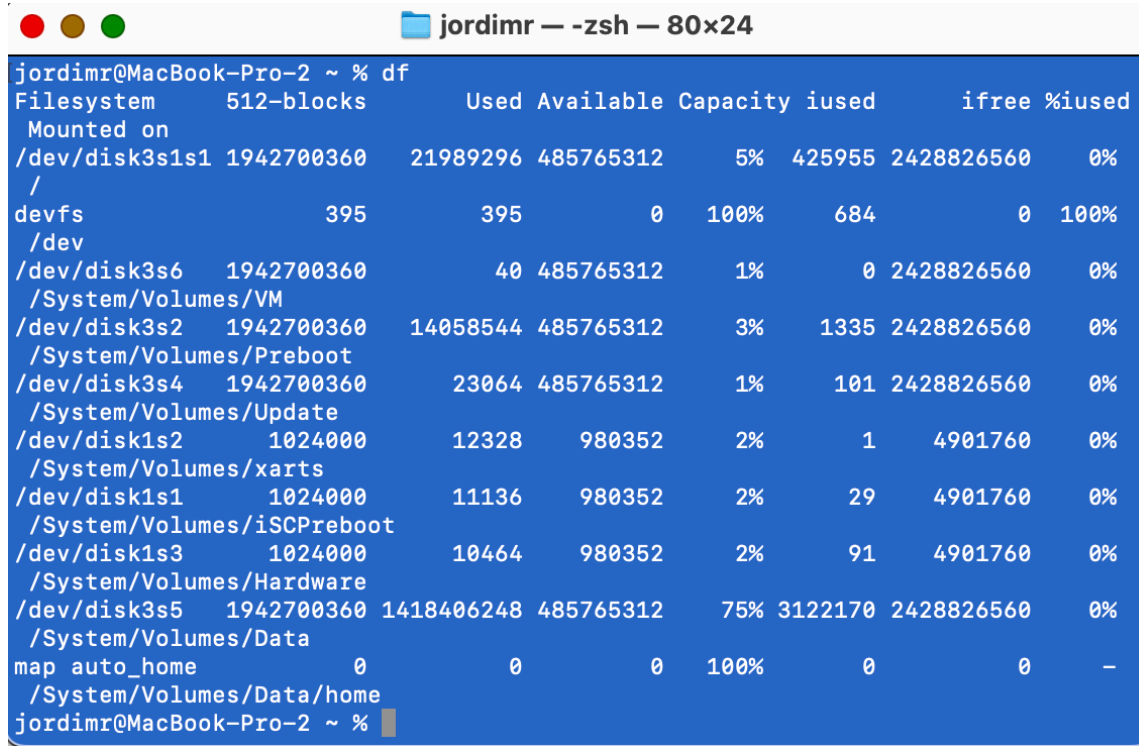
13. **ps x**: list of (labeled) processes running...



```
jordimr — zsh — 80x24
3727 ?? Ss 0:00.03 /System/Library/Frameworks/ImageCaptureCore.framework
3747 ?? Ss 0:00.03 /System/Library/Frameworks/WebKit.framework/Versions
3815 ?? Ss 0:00.01 /System/Library/Frameworks/ApplicationServices.frame
3817 ?? S 0:00.18 /System/Library/PrivateFrameworks/AccessibilitySuppo
3929 ?? Ss 0:00.06 /System/Library/Frameworks/CoreGraphics.framework/Ve
3997 ?? S 0:14.97 /Applications/Keynote.app/Contents/MacOS/Keynote
3998 ?? S 0:00.03 /System/Library/PrivateFrameworks/HelpData.framework
4002 ?? S 0:00.18 /System/Library/Frameworks/StoreKit.framework/Suppor
4004 ?? Ss 0:00.01 /System/Library/Frameworks/AudioToolbox.framework/XP
4007 ?? Ss 0:00.05 /System/Library/Frameworks/Metal.framework/Versions/
4008 ?? Ss 0:00.03 /System/Library/Frameworks/Metal.framework/Versions/
4009 ?? Ss 0:00.38 /System/Library/Frameworks/AppKit.framework/Versions
4010 ?? Ss 0:00.03 /System/Library/Frameworks/QuickLookUI.framework/Ver
4026 ?? Ss 0:00.01 /System/Library/Frameworks/NetFS.framework/Versions/
4084 ?? S 0:00.29 /System/Applications/Utilities/Terminal.app/Contents
4090 ?? Ss 0:00.02 /System/Library/Frameworks/Metal.framework/Versions/
4091 ?? Ss 0:00.02 /System/Library/Frameworks/Metal.framework/Versions/
4093 ?? S 0:00.08 /System/Library/Frameworks/CoreServices.framework/Fr
4094 ?? S 0:00.19 /System/Library/Frameworks/CoreServices.framework/Fr
4095 ?? S 0:00.23 /System/Library/Frameworks/CoreServices.framework/Fr
4085 s000 Ss 0:00.01 login -pf jordimr
4086 s000 S 0:00.02 -zsh
4108 s000 R+ 0:00.01 ps x
jordimr@MacBook-Pro-2 ~ %
```

14. **kill -9** [label]: *force* kill of such [label] process

15. **df**: disc occupancy



A terminal window titled "jordimr — -zsh — 80x24" showing the output of the `df` command. The output is a table with columns: Filesystem, 512-blocks, Used, Available, Capacity, iused, ifree, and %iused. The rows list various filesystems and their usage statistics.

Filesystem	512-blocks	Used	Available	Capacity	iused	ifree	%iused
Mounted on							
/dev/disk3s1s1	1942700360	21989296	485765312	5%	425955	2428826560	0%
/							
devfs	395	395	0	100%	684	0	100%
/dev							
/dev/disk3s6	1942700360	40	485765312	1%	0	2428826560	0%
/System/Volumes/VM							
/dev/disk3s2	1942700360	14058544	485765312	3%	1335	2428826560	0%
/System/Volumes/Preboot							
/dev/disk3s4	1942700360	23064	485765312	1%	101	2428826560	0%
/System/Volumes/Update							
/dev/disk1s2	1024000	12328	980352	2%	1	4901760	0%
/System/Volumes/xarts							
/dev/disk1s1	1024000	11136	980352	2%	29	4901760	0%
/System/Volumes/iSCPreboot							
/dev/disk1s3	1024000	10464	980352	2%	91	4901760	0%
/System/Volumes/Hardware							
/dev/disk3s5	1942700360	1418406248	485765312	75%	3122170	2428826560	0%
/System/Volumes/Data							
map auto_home	0	0	0	100%	0	0	-
/System/Volumes/Data/home							

16. **less** *file*: reading *file*. Quit with Esc + **:q**

17. **cat** *file*: listing last part of *file*

18. **man**: call to manual - guide -help pages

Example: 'man cat'

Text editors:

1. Vim (Vi improved): operates in two modes: command mode and insert mode
2. GNU Emacs (X window system)
3. Kedit, Gedit, Xedit, Kile, etc.

YOU should test them... 🙄

To have a look: online mini-course <http://www.charmm-gui.org/?doc=lecture&module=unix&lesson=1>

(In case you use *Windows*, the first unit starts at the next slide)

2. Basics of FORTRAN



High level language for scientific purposes. Not case sensitive.

Widely used at Barcelona Supercomputing Center!

Why using it here? The simplest to learn/master in short term

Previous step: install a FORTRAN compiler!

Linux (plain way): `> sudo apt-get update`
`> sudo apt-get install gfortran`

Windows 10: For instance, try <https://simply-fortran.en.softonic.com>

Alternatives: (Windows, Mac, Linux): <https://gcc.gnu.org/wiki/GFortranBinaries>

General use:

Optional

Name of a program file: `program.f`

Compiling: `gfortran -O3 -o program.o program.f`

Execution: `./program.o`

2. Basics of Fortran

General Structure: Program units

A. The **program**:

```
PROGRAM myprog  (optional)
```

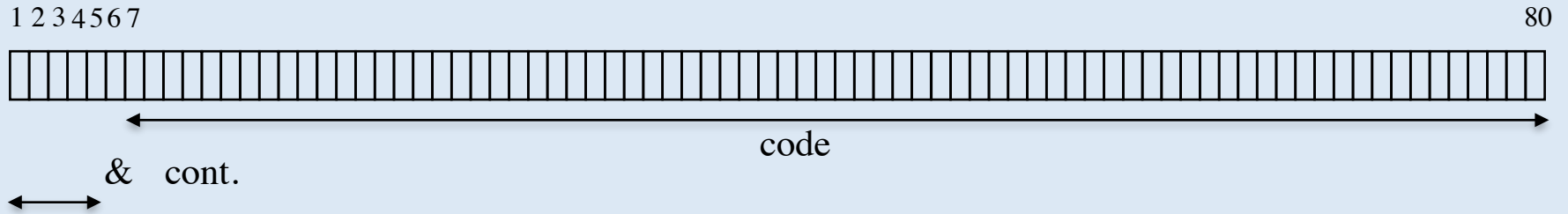
non-executable statements

executable statements

```
END PROGRAM myprog  (optional)
```

B. Other units: **functions, subroutines and modules.**

The “grid”: where to write code and how



numerical labels

```
C*****  
C***** ! This program is useless !  
C*****
```

```
PROGRAM myprog
```

```
INTEGER :: i,j
```

```
IMPLICIT DOUBLE PRECISION (A-H,O-Z)
```

```
do i=1,10
```

```
    if (i.gt.10) goto 200
```

```
    j = 3*i
```

```
    k = 1.15*cos(2.D0*j) + 3.75D0*sin(0.7*j)/tanh(i/j)+7.1D0*sin
```

```
& (datan(1.d0))
```

```
WRITE(20,*) j
```

```
enddo
```

```
200 CONTINUE
```

```
STOP
```

```
END
```

A. All program units must terminate with an END keyword.

Alternatively, a STOP statement causes execution to cease. STOP may be followed by a message to indicate why or where the termination took place.

```
STOP
```

or

```
STOP "Divide by zero attempted. Execution terminated in subroutine  
DIV."
```

Variables are the fundamental building blocks of any program. In Fortran, a variable name may consist of up to 31 alphanumeric characters, of which the first character must be a letter. Underscores allowed, but not spaces or special characters.

INTEGER: 137, -11, 0, etc. Without commas.

REAL: single-precision floating-point number (up to 8 decimal digits) 19., 3498.12, 6.0221415e23, -13.01

DOUBLE PRECISION: up to 16 decimal digits. 15d00, 79.375d-11, 27.8734567807d3, 0.1431573145146599d-120

COMPLEX: two REAL. (1.0,0.0), (-270.,15.5), (0.0,-10.)

CHARACTER: between commas. 'The answer is 42.', 'All the world's a stage.'

LOGICAL: .true. or .false.

In Fortran, **variables must be declared**:

```
INTEGER (optional ::) i,j
```

```
REAL r,s,t
```

```
COMPLEX z,w
```

A variable can be assigned a value at its declaration (a counter, for instance):

```
INTEGER i=2,j
```

but if the keyword `parameter` is included, the assigned value cannot be changed subsequently (an attempt to do so will generate an error):

```
INTEGER, PARAMETER i = 2
```

```
CHARACTER (LEN=16) a
```

LEN indicates the length of the character

Implicit variables can be declared:

```
IMPLICIT DOUBLE PRECISION (A-H,O-Z)
```

Variable Operations

1. With integers:

(-) K : negation

Arithmetic operations (+ - * /) : (addition, subtraction, multiplication, division). The result of these binary arithmetic operations is another integer; in the case of integer division, only the quotient is returned.

NM : N raised** to the M th. power, where M is integer

2. With floating point variables:

Same as above

xy : x raised** to the yth power. Normally evaluated using logarithms, it is slower than raising a floating-point number to an integer.

3. With logical operators

.not. : **negation**

.and. : **intersection**

.or. : **union**

4. With relational operators

.eq. : **equality**

.ne. : **non-equality**

.lt. < : **less than**

.gt. > : **greater than**

.le. <= : **less than or equal to**

.ge. >= : **greater than or equal to**

Expressions, Statements, and Assignments

Expression: combination of symbols that forms a valid unit for evaluation or computation.

```
x + 1.0  
97.4d0  
sin(y)  
x*aimag(z+w)
```

Statement: complete instruction. It starts in column 7

Normally terminated by the end-of-line marker. If a statement is too long for a line, it may be continued by ending the line with an ampersand (&), placed in col.6. The standard line length before a continuation is required is 80 characters.

(starts in column 7)

↓
z = sin(y)

a = b+c/d

do i=1,10

(column 6)
↙ var1 = qa * (qb**3 * (az + bz)) + qc * (qd**2 * (ax - bx))
&
+ qe * (wa + cos(wb))

spaces ("whitespace") are ignored.

A statement may carry a label (starting in col.1) that uniquely identifies it by a number. The number must consist of one to five digits, at least one of which must be nonzero. FORMAT statements must be labeled.

(starts in column 1)
↓
100 continue
2100 format(6i)

(column 7)
↖

↓
Integer 3I12
Floating-Point 6F8.2
Character A12

Assignment statements: assign an expression to a quantity. Uses the equals sign (=), but this symbol does not have the same meaning as it has in mathematics. In Fortran, the statement

$x = x + 1.0$

means that **the value x is to be incremented by 1 and the result stored into the memory location reserved for the quantity represented by x .**

However, the statement

$x + 1.0 = y$

is **not** a valid assignment in Fortran.

Intrinsics

Functions defined by the compiler; Fortran provides a wide range of mathematical functions (exp, log, sin, cos, etc.) See section 2.4 of “SCIENTIFIC PROGRAMMING IN FORTRAN 2003”, K.Holcomb, 2012).

DATA statements used to initialize a variable or group of variables.

```
DATA a,b,c / 1.,2.,3./  
DATA a,b,c / 1.,2.,3. /, x,y,z / 4.,5.,6. /  
DATA ra / 100*1.0 /
```

The last form illustrated above initializes a 100-element array to one at compile time. Individual elements or sections may also be initialized in a similar manner. In most cases, the `data` statement can be eliminated by initializing variables in their declarations.

```
REAL a=1.,b=2.,c=3.  
REAL, DIMENSION(100) ra=(/100*1.0)/
```

Practice 1: Learning Fortran

1. Write a **full program** that adds one integer with one real. Be sure to use correct style (program name, no implicit typing, correct declarations). Add the user to enter the two numbers. Print the result in an **external file**.

Note: The code **and** the result must be printed. Include all details in your **report of Practice 1**, indicating each part and, eventually, commenting about relevant details (precision used, format, etc.)

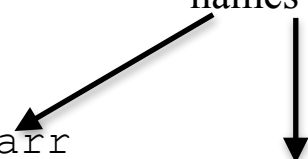
IMPORTANT: The report of each Practice must be delivered in PDF format

Array is a variable that consists of more than one element, each of which is of the same type. It has at least one dimension.

Size of the array = total number of elements, i.e. the size of an $N \times M$ array is N times M

Shape of an $N \times M$ array is (N, M)

names of arrays



```
INTEGER, DIMENSION(10)      iarr
REAL (rk), DIMENSION(200,200)  rarr, darr
CHARACTER(LEN=12), DIMENSION(4) carr
COMPLEX, DIMENSION(10,5,3)    zarr
LOGICAL, DIMENSION(200)      larr
```

A file may be **opened** with the statement

```
OPEN([UNIT=]un, FILE=fname)
```

```
open(12, file="data.in")
```

```
open(unit=33,file="text.txt")
```

```
open(19,file="unit.dat")
```

```
open(28,file="input.dat")
```

ALERT!!! File number 6 reserved to screen

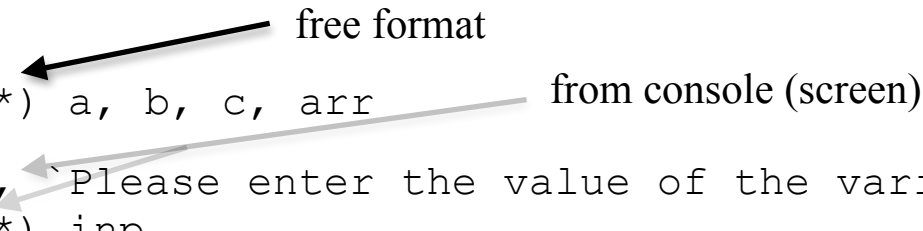
Reading and Writing Data

The unit number is used in input-output statements to reference the file.

```
READ(UNIT=iu, options) varlist
```

```
WRITE(UNIT=iu, options) varlist
```

```
READ(5,*) a, b, c, arr
PRINT *, 'Please enter the value of the variable inp:'
READ(*,*) inp
...
WRITE(10,*) a, b, c, arr
WRITE(*,*) a, b, c, arr
```



```
PRINT ' (20X,I5,20X,I5/21X,I4,21X,I4)', N, M, I, J
```

causes 20 spaces to be skipped, an integer written, another 20 spaces skipped and another integer written; then on the next line 21 spaces are skipped and an integer written, and finally 21 spaces and an integer are output.

Format Statements

Edit descriptors must be used in conjunction with a PRINT, WRITE, or READ statement.

Normally it suffices to enclose the format in single quotes and parentheses and provide it as an argument to the keyword

```
print '(I5,5F15.5)', i, a, b, c, d, e
write(10, '(8E20.7)') arr
read(in_unit, '(2a)') c1, c2
```

If the same format is to be used repeatedly or it is complicated, the FORMAT statement can be used. This takes the form

```
label FORMAT(formlist)
```

The FORMAT statement must be labeled.

(grid) 1 2 3 4 5 6 7

```
10  FORMAT(I5,5F15.5)
3100 FORMAT(8E20.7)
```



Conditionals

Conditionals are used to determine whether one path or another should be taken. The `IF` and `IF-THEN-ELSE` statements implement this logic.

```
IF ( condition ) statement
```

or

```
IF ( condition ) THEN
    statement1
    statement2
    etc.
ENDIF
```

or

```
IF ( condition ) THEN
    actions if true
ELSE
    actions if false
ENDIF
```

For instance:

```
IF ( a .lt. b ) THEN
    z = b-a
ELSE
    z = a-b
ENDIF
```

```
IF ( x.eq.y ) THEN
    w = x**2
ELSE
    w = x*y
ENDIF
```

Loops

Much of programming consists of performing a group of operations many times on different data, i.e. iterating over the data. The primary loop construct in Fortran is DO.

```
DO lv = lexpr, uexpr, sexpr
    code
END DO
```

or

```
name: DO lv = lexpr, uexpr, sexpr
    code
END DO name
```

lv, lexpr, uexpr, sexpr must be scalar integer variables.

Example:

```
do k = 1, kmax
    do j = 1, jmax
        do i = 1, imax
            a(i,j,k) = b(i,j,k)
        enddo
    enddo
enddo
```

Implied DO

The implied do is a construct that is used in input/output statements to give more control over the output of an array. It can also be used to initialize an array to constant values. It takes the general form

```
(var, iterator=lexpr, uexpr, sexpr)
```

For instance:

```
WRITE(15,*) (A(I), I=1,10)
READ(10,100) ((R(I,J), J=1,10), I=1,20)
PRINT *, ((S(I,J), I=1,20), J=1,10)
DATA (((A(I,J,K), K=1,10), J=1,10), I=1,10) / 50. /
```

DO WHILE

The DO WHILE loop executes for as long as a condition is .true.

```
DO WHILE ( condition )
    code
END DO
```

It is critical that the code within the loop provide a stopping condition for the loop; otherwise an infinite loop will result.

Practice 1: Learning Fortran

2. Use the Leibniz formula to compute π

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

Compute until your result agrees to at least five decimal digits within the correct value (3.1415926535897932384626433....) This series converges very slowly so this can take a lot of iterations...**Display a graph** showing the evolution with k , i.e. $\pi(k)$ as a function of k .

Note: The code **and** the result must be printed. Include all details in your **report of Practice 1**, commenting about relevant details (precision used, format, etc.) if necessary.

B. Subroutines and Functions

Different groups of operations may be cast into the form of subroutines and functions.

Functions take zero or more arguments and return exactly one result

Subroutines take zero or more input arguments and can return more than one result in the form of output arguments.

The `IMPLICIT NONE` statement that is recommended must appear in every program unit, including all subroutines and functions.

The function type may also be declared inside the function. This is the required form if more than one attribute is needed, such as array size.

```
function xy(w)
real, dimension(100) xy
```

A function ends with an `END FUNCTION` statement.

A **FUNCTION** is declared

```
<type> FUNCTION name(arglist)
```

Just like variables, a function is typed. The type corresponds to the return value. Remember that a function may return only one variable. The return variable is the name of the function itself.

Examples

```
real function myfunc(x,y)
integer function index(a,n)
logical function is_imag(z,w)
```

Example: Use of a function

PROGRAM FUNDEM

C Declarations for main program

REAL A,B,C

REAL AV, AVSQ1, AVSQ2

REAL AVRAGE

C Enter the data

DATA A,B,C/5.0,2.0,3.0/

C Calculate the average of the numbers

AV = AVRAGE(A,B,C)

AVSQ1 = AVRAGE(A,B,C) **2

AVSQ2 = AVRAGE(A**2,B**2,C**2)

PRINT *, 'Statistical Analysis'

PRINT *, 'The average of the numbers is:', AV

PRINT *, 'The average squared of the numbers: ', AVSQ1

PRINT *, 'The average of the squares is: ', AVSQ2

END FUNDEM

C Define the function

REAL FUNCTION AVRAGE(X,Y,Z)

REAL X,Y,Z,SUM

SUM = X + Y + Z

AVRAGE = SUM /3.0

RETURN

END

A `SUBROUTINE` is declared

```
SUBROUTINE name(varlist)
```

The list of variables is called the parameter list. For example, `SUBROUTINE bvals(a,b,n)`

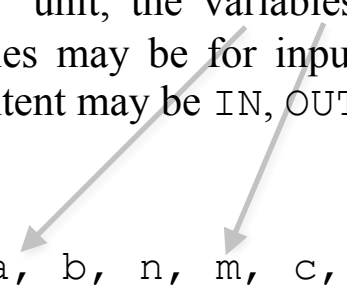
Subroutines may return multiple values and thus do not take a type. They are invoked with the `CALL` statement: `CALL name(varlist)`.

Variables in the `CALL` parameter list are **actual** arguments. The variables in the parameter list of the subroutine definition are **dummy** arguments. The names of the actual and dummy arguments need not be the same, but they must match in number and type.

Subroutines must end with an `END SUBROUTINE name` statement.

Just as in the program unit, the variables must be declared appropriately. In the case of subroutines, those variables may be for input, output, or both. This may be indicated by the `INTENT` attribute. The intent may be `IN`, `OUT`, or `INOUT`.

Example



```
SUBROUTINE mysub(a, b, n, m, c, d)
REAL, DIMENSION(N), INTENT (IN)  a, b
INTEGER,          INTENT (IN)    n, m
REAL, DIMENSION(M), INTENT (INOUT) c
REAL, DIMENSION(M,N), INTENT (OUT) d
```

Variables declared `INTENT (IN)` may not be changed within the subroutine.

Example: Use of subroutines

Plain code:

```
program integration1

implicit none

integer  :: i, j
real(8)  :: x, y, integ

integ = 0.0D0

do j=1,10
  y = dble(j)*1.0D0
  do i=1,10
    x = dble(i) * 2.5D-1
    integ = integ + sin(x+y)
  enddo
enddo

write (*,'(a,e20.10e3)') ' Integration value = ',integ

end program integration1
```

```

program integration
implicit none

integer i, j
real*8 x, y, integ
real*8 val

integ = 0.0D0

do j=1,10
  y = dble(j)*1.0D0
  do i=1,10
    x = dble(i) * 2.5D-1
    call subr(x,y,val)
    integ = integ + val
  enddo
enddo

write (*,'(a,e20.10e3)') ' Integration value = ',integ

end program integration

subroutine subr(a,b,c)
implicit none
real*8 :: a,b,c
c = sin(a+b)
return
end subroutine subr

```

Dummy variables!

Practice 1: Learning Fortran

3. Use subroutines and/or functions to write a code to compute the Fourier integral of a given time-dependent correlation function such as $C(t) = \langle A(0) \cdot A(t) \rangle$. For instance, in molecular water simulations, spectral lineshapes $I(\omega)$ are usually found from

$$I(\omega) \sim \frac{2Nq^2}{\pi\omega^2} S(\omega) \equiv \frac{2Nq^2}{\pi\omega^2} \int_0^\infty dt \langle \vec{v}_{H_i}(t) \vec{v}_{H_i}(0) \rangle \cos(\omega t)$$

Use the same idea to obtain **S(w)** for C(t), given that it is read from a known file, say file named “corr.dat” (corresponds to an oxygen velocity autocorrelation), with the usual two column format (t, C(t)). You’ll find the file at “ATENEA” database. Note that the time step used in “corr.dat” is of 0.0005 ps.

Find:

$$S(\omega) = \int_0^\infty dt C(t) \cdot \cos(\omega t)$$

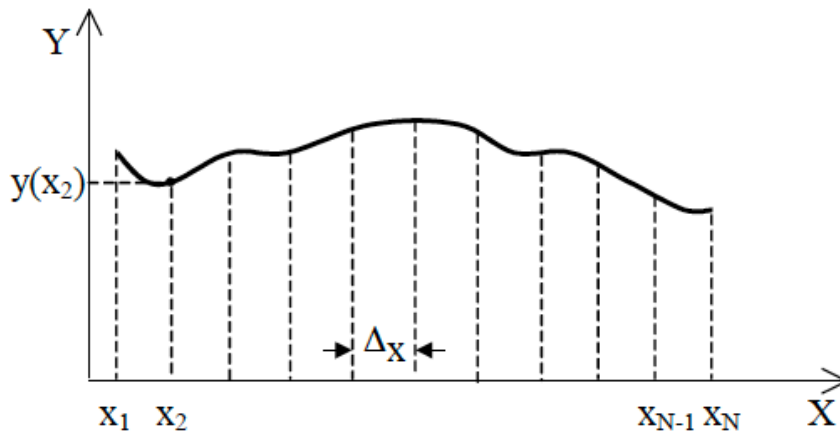
Note: The code **and** the result (a plot of S(w) as shown below, page 34) must be printed. Include all details in your **report of Practice 1**, commenting about relevant ideas/procedures you used, if necessary.

See additional indications! ->

In order to compute the numerical integral, use a **quadrature or Simpson method**. Here is a basic survey:

Basic integration methods

Numerical integration is a form of resolution of integrals that can be applied, in general, in the vast majority of cases. It is about considering functions that are described by a set of points (we will assume that we are in 2D, to simplify) so that one has, symbolically, a table of type: $y(x_1, \dots, x_N)$. Note that we consider equi-space of width Δx , so $x_i = x_1 + \Delta x(i-1)$.



Suppose we want to calculate the integral of $y(x)$ between x_1 and x_N :

$$S = \int_{x_1}^{x_N} dx y(x)$$

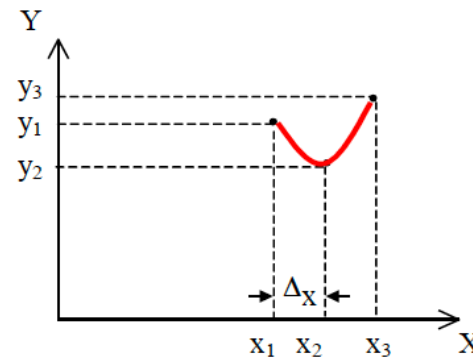
The meaning of integration is that S is the (continuous) sum of the function $y(x)$ between x_1 and x_N . Corresponds to the area bounded by the curve and the X -axis between the two integration limits. Example: finding the area of a rectangle is trivial!

There are many ways to numerically calculate these integrals. The most simple are **squaring** (trapezoidal rule) and **Simpson** formulas.

Squaring (“quadrature”): This is the search for the area under the curve taking the value of the function in an interval as the sum of partitions (of the interval) of the average of the two extreme values of each partition, i.e.

$$\int_{x_1}^{x_3} y(x) dx = \sum_{\text{Toutes les partitions}} \Delta x \left[\frac{y_i}{2} + \frac{y_{i+1}}{2} \right] + O((\Delta x)^3 y''') =$$

$$= \Delta x \left[\frac{y_1}{2} + \frac{y_2}{2} \right] + \Delta x \left[\frac{y_2}{2} + \frac{y_3}{2} \right] + O((\Delta x)^3 y''')$$



This formula is exact for polynomials of degree 1. If we want more precision, we need to go to exact formulas up to higher degree polynomials.

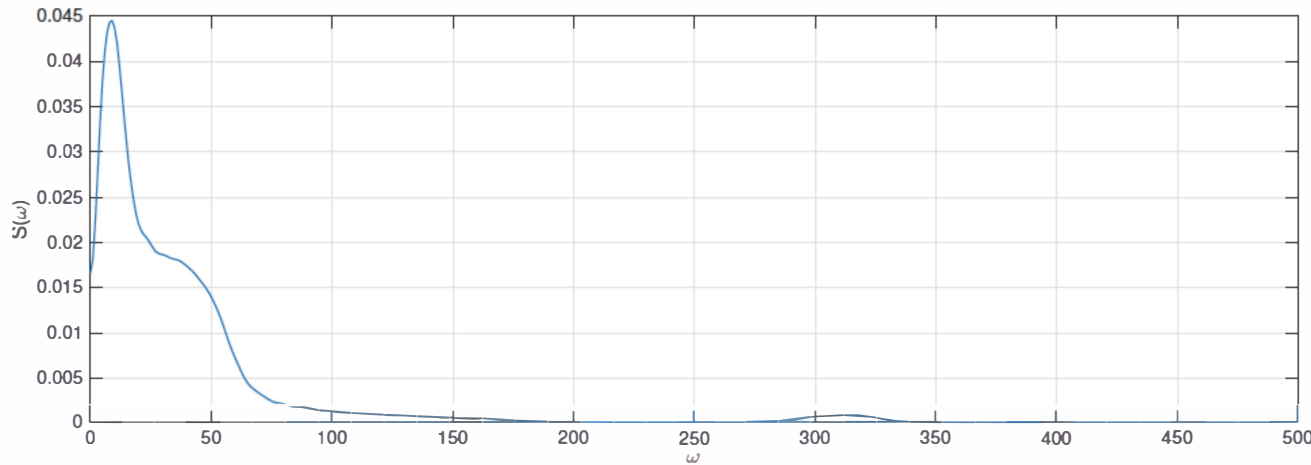
Simpson's formulas: The calculation is done considering 3, 4 or more points. With 3 we have the "classic" Simpson formula and with 4 points, the formula 3/8. Respectively, they are:

$$\int_{x_1}^{x_N} y(x) dx = \sum_{\text{Partitions "i"}} \Delta x \left[\frac{y_{i-1}}{3} + \frac{4y_i}{3} + \frac{y_{i+1}}{3} \right] + O((\Delta x)^5 y^{(4)})$$

and it is exact up to third order. Rule 3/8, also exact until third order is given by:

$$\int_{x_1}^{x_n} f(x) dx = \sum_{k=1}^{(n-1)/3} \int_{x_{3k-2}}^{x_{3k+1}} f(x) dx = \sum_{k=1}^{(n-1)/3} \frac{h}{8} (3f(x_{3k-2}) + 9f(x_{3k-1}) + 9f(x_{3k}) + 3f(x_{3k+1})) + O(h^5 f^{(4)}(x)) \quad (13)$$

with the latter, the expected result of the calculation for section C is:



Hint: Choose a set of $\omega \in [0, 500]$ ps⁻¹, since time t is given in picoseconds

where:

$$f(t) = C(t) \cdot \cos(\omega t)$$

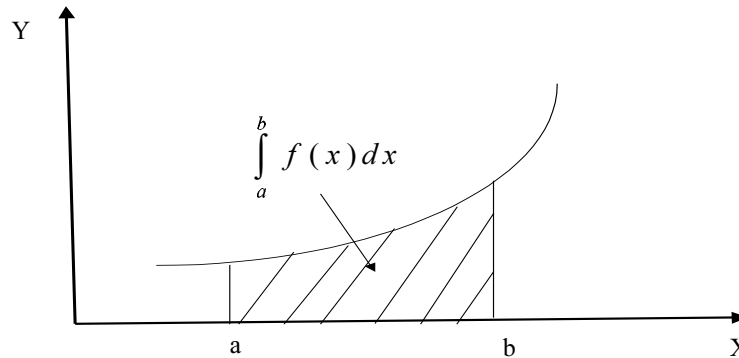
and $\Delta t = 0.0005$ ps

Practice 1: Learning Fortran

4. Compute integrals with the MC method, using random sampling.

Calculation of a surface

Idea: Use of the mean value theorem for integrals $\int_a^b dx f(x) = (b - a) \langle f \rangle$



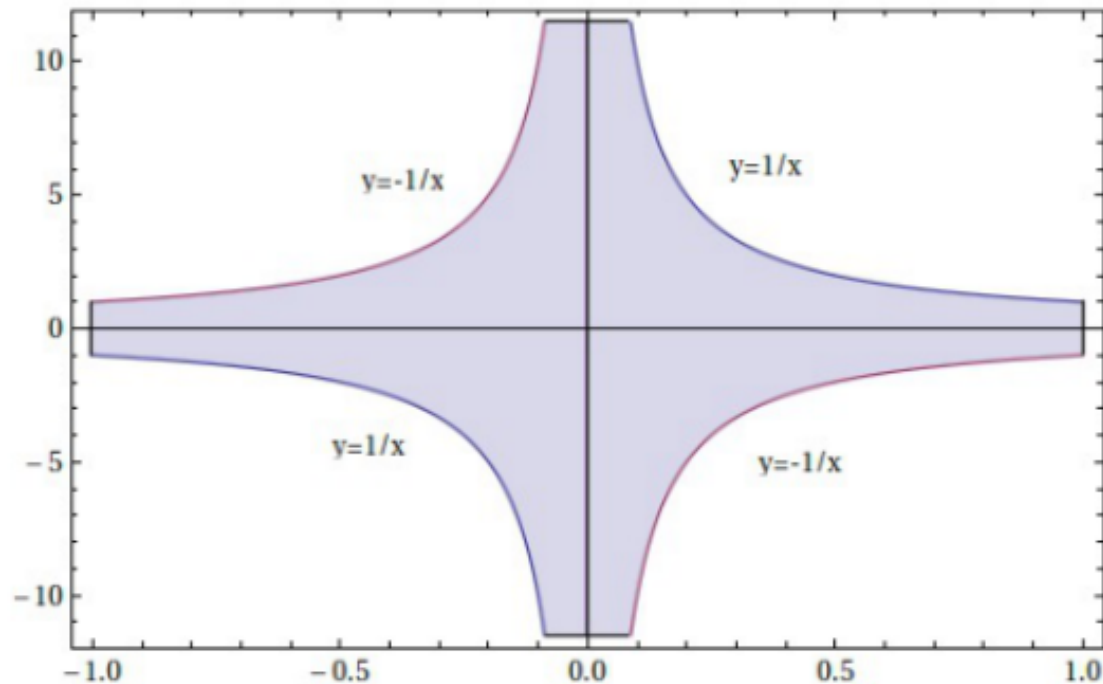
If we get $\langle f \rangle$ as

$$\langle f \rangle_N = \frac{1}{N} \sum_{i=1}^N f(x_i)$$

The value of the integral is:

$$\int_a^b dx f(x) = (b - a) \frac{1}{N} \sum_{i=1}^N f(x_i)$$

Evaluate the area enclosed between the curves $y = x^{-1}$ and $y = -x^{-1}$ within the range $x \in [-1, 1]$ and $y \in [-11, 11]$, represented in the figure. Note that the exact value is $A_{\text{exact}} = 13.5916$

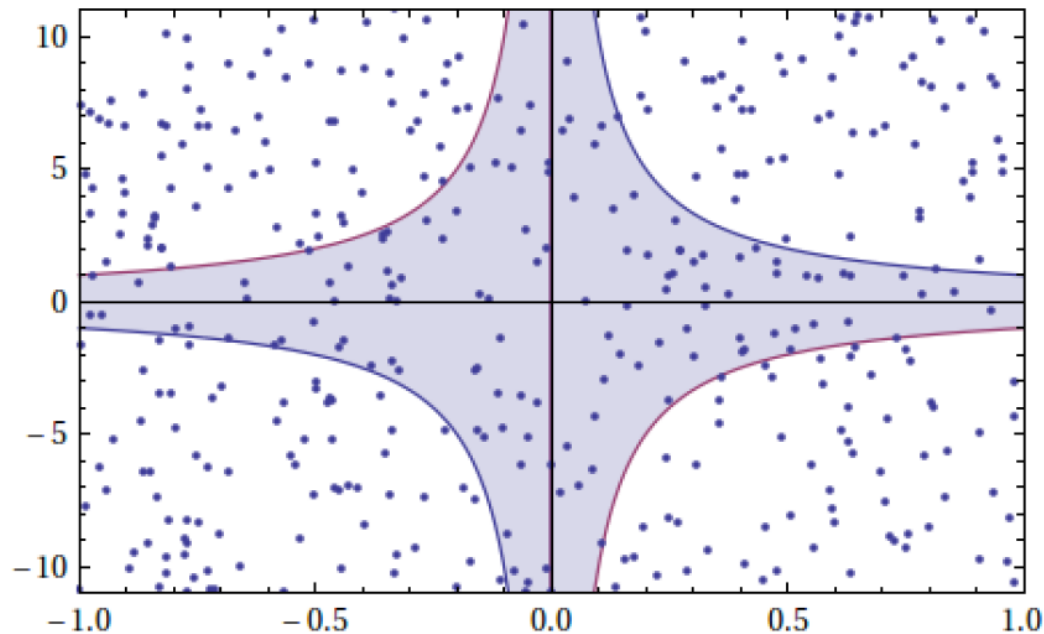


Procedure:

A. Generate N points (x_i, y_i) at random.

x_i belong to a uniform distribution in the range $[-1,1]$

y_i belong to a uniform distribution in the range $[-11,11]$

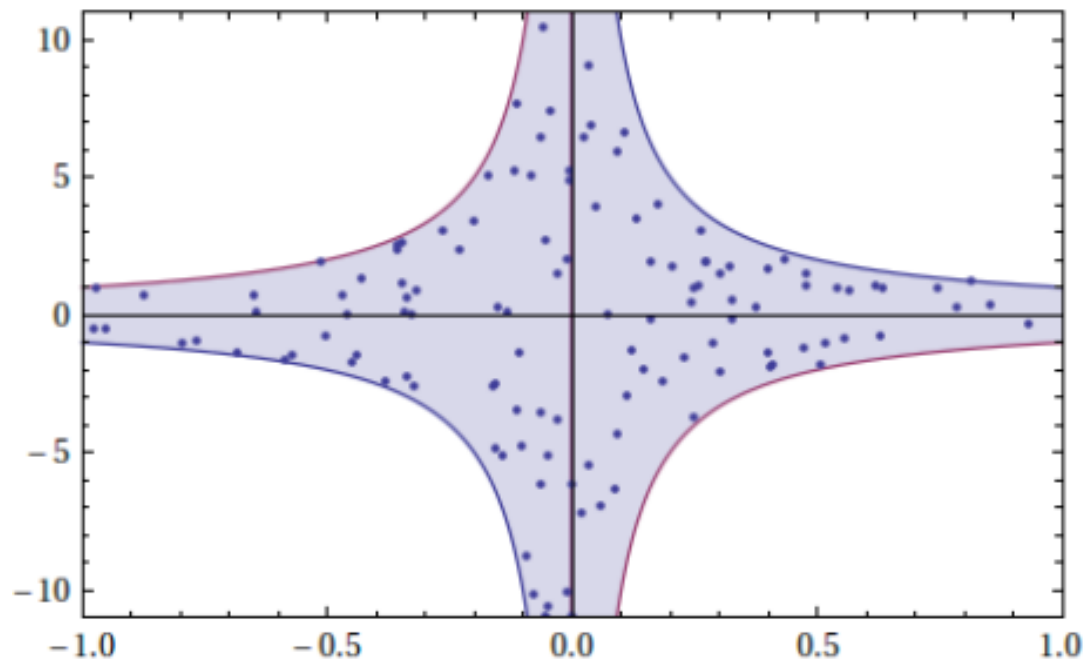


Hint: You'll need a good random number generator!
Try "RANDOM_SEED", works in most of Fortran versions

B. Calculate the number of points N_{hit} inside the region of interest

and estimate the area as follows:

$$A(N) = 2 \times 22 \times \frac{N_{hit}}{N}$$



C. Run a number of different MC series (for instance, 3) with $N = \{10^2 - 10^7\}$

For each N, calculate the mean value of the area:

$$\langle A(N) \rangle = \frac{1}{M} \sum_{i=1}^M A_i(N)$$

and the standard deviation σ (variance defined as σ^2)

$$\langle \sigma(N) \rangle = \sqrt{\langle (A(N) - A_{exact})^2 \rangle}$$

D. Finally **plot $A(N)$ and $\sigma(N)$**

Note: The code **and** the results must be printed. Include all in your **report of Practice 1**, commenting about relevant details if necessary, such as how many MC series have you employed for each N, what parameters are acceptable for the computational cost, etc.

Practice 1: Integration algorithms (to do after the corresponding theory lesson)

5. Consider an harmonic oscillator of mass $m = 200 \text{ g}$ and force constant $k = 2 \text{ N/m}$. Integrate numerically the equation of motion and compare the results of positions and velocities with the exact analytical solution. Compute also the kinetic, potential and total energies.

Initial position and velocity: $x_0 = 5 \text{ cm}$, $v_0 = 0$. Integration algorithms and timesteps:

Euler, $\Delta t = 0.001\text{s}$, $\Delta t = 0.01\text{s}$, $\Delta t = 0.02\text{s}$

Euler predictor-corrector, $\Delta t = 0.02\text{s}$

Verlet, $\Delta t = 0.02\text{s}$

Note: The code **and** the results must be printed. Include all in your **report of Practice 1**, in particular you must compare **in the same graph** “exact” vs. “algorithms” (Euler, predictor-corrector and Verlet) and report all three energies, if possible in the same graph. Use different time steps and show the error between “exact” vs. “algorithms” solutions.